Cisco Comply to Connect (C2C) submit2Cmrs Automated Reporting Guide

---

Table of Contents

- Cisco Comply to Connect (C2C) submit2Cmrs Automated Reporting Guide
  - Table of Contents
- 1. Copy Files to Reporting Application Server
- 2. File Paths and Submission Mode
- 3. Publisher and Organizational Metadata
- 4. API Connectivity and Security
- 5. Operational Parameters
- 6. Compliance Rule Mapping
- 7. Recurring Automatic Submission to CMRS
  - Linux: Using cron jobs
  - Windows: Using Task Scheduler

This script automates the reporting of Comply-to-Connect (C2C) endpoint data. It transforms raw device information (exported from the Cisco C2C 3.0 Reporting Tool) into a standardized XML format required by the Department of Defense (DoD) for continuous monitoring and risk scoring (CMRS).

---

1. Copy Files to Reporting Application Server

Copy both `cmrsCustomerData.py` and `submit2Cmrs.py` files to the Search Head server on your environment where the Cisco C2C Reporting Application 3.0 is running.

Recommended Paths:

- **Linux (User Home):** `/home/<user>/`
- **Windows (User Documents):** `C:\Users\<user>\Documents`

---

2. File Paths and Submission Mode

The `cmrsCustomerData.py` file acts as the central configuration point for the reporting script. It defines how the script locates data, identifies the reporting entity to the DoD, and connects to the DISA CMRS API.

These settings determine where the script pulls raw data and whether it sends that data directly to an API or saves it to a local file.

Variable	Description	Default

Variable	Description	Default
`c2cReportPath`	The absolute path to the CSV output from the reporting tool (e.g., Splunk lookup).	`/opt/splunk/.../cisco_catalyst_analytics_reports.csv`
`offlineUpload`	Set to `True` to generate a local XML file; `False` to attempt direct SOAP API submission.	`False`
`offlineReport`	The filename/path for the generated XML when `offlineUpload` is enabled.	`./offlineCMRSReport.xml`

3. Publisher and Organizational Metadata

This section identifies your specific installation and organizational hierarchy to the DISA CMRS system. These values are typically assigned during your CMRS registration.

Variable	Description	Default/Example
`publisherName`	The FQDN or unique identifier for your reporting server.	`test.ciscosecuritylab.com`
`publisherVersion`	The version of the reporting application.	`3.0.0`
`iseVersion`	The current version of Cisco ISE in your environment.	`3.4.3`
`iseSerial`	The serial number or unique deployment ID for Cisco ISE.	`12345ABCDE`
`reportingOwnOrg`	DoD-assigned ID for the Owning Organization.	`12345`
`reportingAdminOrg`	DoD-assigned ID for the Administering Organization.	`12345`
`reportingGeolocation`	DoD-assigned location ID for the reporting sensor.	`12345`

4. API Connectivity and Security

These variables define the secure connection to the DISA CMRS SOAP endpoint.

- **SOAP Endpoint:** `https://example.com/soap/submit`
- **Authentication:** The script uses Mutual TLS (mTLS). Provide the paths to your PEM-encoded certificates.
  - `certFile`: Your client certificate.
  - `keyFile`: Your private key.
  - `caBundle`: The CA certificate used to verify the DISA server.
- **SOAP Action:** `SubmitRecords` (This value is static and should not be modified).

---

5. Operational Parameters

Settings used to tune the performance and logic of the reporting script.

- **`reportingBatchSize`**: Defines how many endpoint records are included in a single XML upload.
  - *Note:* The theoretical maximum is ~250 endpoints per SOAP Submit; a size of 200 is recommended for stability.
- **`acasDelta`**: The frequency (in days) of expected ACAS scans. If an endpoint has not been seen by ACAS within this window, it will be marked as "Failed" for the vulnerability scan requirement.

---

6. Compliance Rule Mapping

These variables map the column headers in your CSV report to the specific C2C rules required by CMRS. If you modify the search macros in your reporting app, update these names to match.

C2C Rule	CSV Column Mapping (Examples)
Firewall	`C2CFirewallResult`
Anti-Malware	`C2CMalwareResult`
Encryption	`C2CEncryptResult`
Patch Agent	`C2CPatchAgentResult`
Patching	`C2CPatchResult`
Endpoint App WL	`C2CAppWlResult`
PKI Roots	`C2CPKIRootsResult`
Endpoint Monitoring	`C2CEDRResult`

> **Implementation Note:** Ensure that the `certFile` and `keyFile` are stored securely with restricted file system permissions. The script requires read access to these files to successfully authenticate with the DISA CMRS gateway.

---

7. Recurring Automatic Submission to CMRS

After the `cmrsCustomerData.py` file has been updated, reporting can be automated via setting up recurring execution of the `submit2Cmrs.py` script on Linux and Windows systems running the reporting application.

Linux: Using cron jobs

Create a cron job to run the Python script at desired intervals. Example cron entry to run every Sunday at midnight:

```
0 0 * * 0 /usr/bin/python3 /home/splunk/submit2Cmrs.py
```

Windows: Using Task Scheduler

1. **Open Task Scheduler:** Search for "Task Scheduler" in the Start menu and open it.
2. **Create a New Task:** Click on "**Create Task…** " in the Actions pane.
3. **General Tab:**
   - Name the task (e.g., "Run submit2Cmrs.py weekly").
   - Optionally, select "**Run whether user is logged on or not**".
   - Check "**Run with highest privileges**" if needed.
4. **Triggers Tab:**
   - Click "**New…** " to create a trigger.
   - Set *Begin the task* to "**On a schedule**".
   - Choose **Weekly**.
   - Select **Sunday** as the day.
   - Set the Start time to **12:00:00 AM** (midnight).
   - Click **OK**.
5. **Actions Tab:**
   - Click "**New…** " to create an action.
   - Set *Action* to "**Start a program**".
   - In *Program/script*, enter the full path to the Python executable (e.g., `C:\Program Files\Splunk\bin\python3.exe`).
   - In *Add arguments (optional)*, enter the full path to your script (e.g., `C:\<UserHome>\submit2Cmrs.py`).
   - Click **OK**.
6. **Conditions and Settings Tabs:** Adjust as needed (e.g., to wake the computer to run the task).
7. **Save the Task:** Click **OK** and enter credentials if prompted.

Ensure that the Python interpreter path and script path are correct and accessible by the Task Scheduler. Also, verify that any required environment variables or dependencies are available in the context where the task runs.